Intro

Definitions
- 1. Acting humanly
  - Turing Test Approach
- 2. Thinking rationally
- 3. Thinking humanly
- 4. Acting rationally

Automation vs AI

Latihan

Definitions

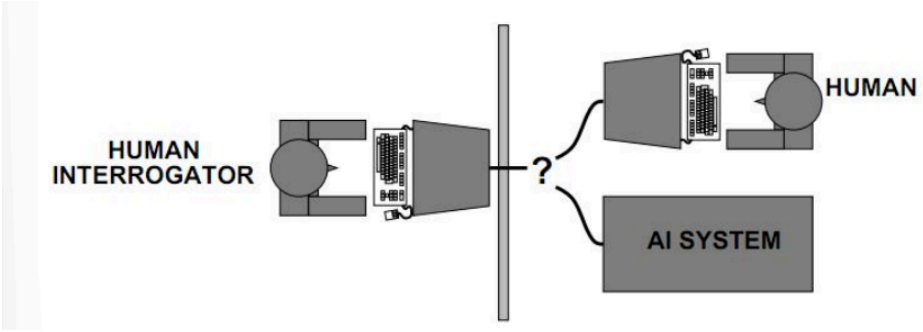
Based on approaches:

1. Acting humanly

"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990)

"The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991)

Turing Test Approach



- A satisfactory operational definition of intelligence
- Pass: human interrogator cannot tell whether the responses come from a person or a computer
- Human can speak, listen, think, see, smell, and feel.
- AI can speak through speaker, listen through microphone, and think with knowledge representation, automated reasoning, and machine learning.

2. Thinking rationally

"The study of mental faculties through the use of computational models."
(Charniak and McDermott, 1985)

"The study of the computations that make it possible to perceive, reason, and act."
(Winston, 1992)

- Meniru proses penalaran manusia (dasar dari pengambilan suatu kesimpulan/keputusan/pemikiran)

- What does it mean to behave intelligently?
- Approach with laws of thought
- Main focus: Reasoning

3. Thinking humanly

"The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense." (Haugeland, 1985)

"[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . ." (Bellman, 1978)

- Meniru proses berpikir manusia
- Programs that operate (internally) the way humans do
    - actual working of human minds
    - Introspection by trying to catch our own thought as they go by
    - psychological experiment by observing a person in action
    - brain imaging by observing the brain in action
- Cognitive modeling approach
    - computer models from AI and experimental techniques from psychology to construct precise and testable theories of human minds
- Machine with minds

4. Acting rationally

"Computational Intelligence is the study of the design of intelligent agents." (Poole *et al.*, 1998)

"AI . . . is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

- Goal, knowledge → act rationally (dasar dari pengambilan suatu tindakan)
    - correct inference is not all of rationality. in some situations, there is no provably correct thing to do but smth must still be done
- Rational Agent → agent that acts so as to achieve the best outcome, or, when there is uncertainty, the best expected outcome

Automation vs AI

Automation	AI
Make the system able to complete work (tasks) in the same way every time	Make the system capable of responding to changes in information/needs from the surrounding environment dynamically; and provide interpretive Decisions
Suitable for tasks that do not require changes over time	Suitable for relatively complex tasks that require making interpretive decisions
Potential to: increase productivity, reduce operational costs, simplify workflow	Potential to: increase productivity, lower operational costs, create intelligent workflows that can respond to dynamically changing needs

Latihan

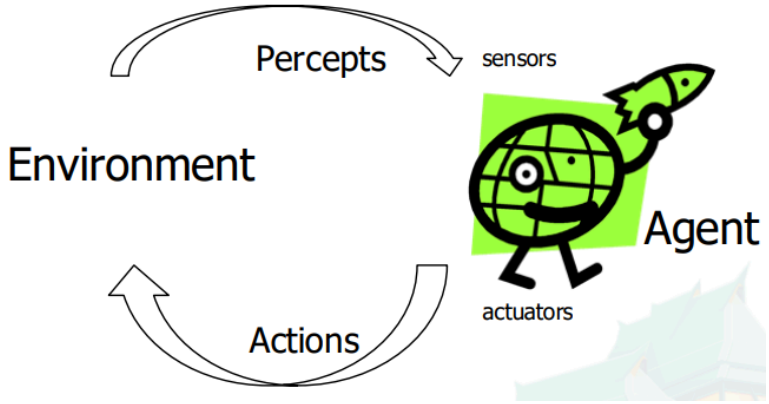
1. Pengenalan gambar dengan menggunakan hierarchical temporal memory yang meniru prinsip biologi dari neocortex merupakan contoh pengembangan ilmu AI dengan pendekatan...
Thinking humanly
2. Dengan berbekal pengetahuan (algoritma) dan tujuan (goal) yang ingin dicapai, sistem mencari rangkaian aksi untuk menghasilkan solusi terbaik, merupakan definisi AI dengan pendekatan...
Acting rationally
3. Pembuatan sistem pakar yang mencoba mengimitasi perilaku pakar dalam mendiagnosis kemungkinan penyakit tanaman berdasar pengamatan terhadap perilaku pakar tersebut merupakan contoh teknologi AI dgn pendekatan...
Acting humanly
4. Sebuah persoalan logic puzzle, diselesaikan dengan pendekatan pencarian memanfaatkan algoritma backtracking, termasuk dalam sistem berbasis AI dengan pendekatan...
Acting rationally
5. Persoalan teka-teki yang diselesaikan dengan menggunakan kaidah-kaidah

inferensi logika (modus ponen, modus tollen, dll) termasuk system AI dengan pendekatan...
Thinking rationally
6. Software akuntansi yang menggunakan berbagai rumus penghitungan rugi laba dengan input berupa file xls dengan struktur kolomnya sudah pasti dan nilai sel berupa angka
non-AI
7. Sistem lampu lalu lintas yang rentang waktu menyalanya lampu merah/kuning/hijau ditentukan berdasar jumlah kendaraan yang lewat dari arah tertentu, dihitung secara otomatis berdasar gambar yang tertangkap kamera cctv di atas lampu lalu lintas tsb.
AI
8. Sistem tanya jawab pemesanan kamar hotel menggunakan flowchart yang sudah ditentukan di dalam program. Pelanggan memberikan input dengan memilih dari kandidat input yang disediakan program.
non-AI
9. Perangkat lampu yang menyala jika ada objek yang berjalan di bawah lampu tersebut. Lampu akan padam jika sudah tidak ada objek bergerak di bawah lampu untuk satu waktu tertentu
non-AI

Intelligent Agent

- Agents
  - Rational Agent
    - Limited Rationality
- PEAS (Performance, Environment, Actuators, Sensors)
  - Examples
  - Properties of Task Environment
- Agent Programs
  - Table-lookup Agent
  - Simple Reflex Agent
  - Model-based Reflex Agent
  - Goal-based Agent
  - Utility-based Agent
  - Learning-based Agent
- AI Agents vs Agentic AI
- Leveling of Agent
  - Problem Solving Agent
  - Knowledge Based Agent
  - Learning Agent

Agents



- sensor → menerima informasi dari lingkungan
- actuators → melakukan aksi thd lingkungan
- **Definition** of an agent:

1. Software that **gathers information** about an environment and **takes actions** based on that information
2. computer system that is situated in some environment, and that is capable of autonomous action in this environment in order to meet its design objectives → **Computational agents that behave autonomously**
Anything that can be viewed as **perceiving** its environment through **sensors** and **acting** upon that environment through **effectors/actuators**.

- "Good behavior" of an AI agent depends on the performance measure → **rational agent**

Rational Agent

- **Performance measure**: An objective criterion for success of an agent's behavior
- For each possible percept sequence, a rational agent should select an action that is expected to **maximize** its **performance measure**, **given** the evidence provided by the **percept sequence** and whatever built-in knowledge the agent has.
  → **A rational agent takes actions it believes will achieve its goals**
- Rationality ≠ Omniscience
  (i.e. Assume the most recent forecast is for rain but I did not listen to it and I did not bring my umbrella. Is that rational? Yes, since idk abt the recent forecast)
- Rationality ≠ Success
  (i.e. Suppose the forecast is for no rain but I bring my umbrella and I use it to defend myself against an attack. Is that rational? No, although successful, it was done for the wrong reason)

- So, we want to use limited rationality ⇒ **acting in the best way** you can subject to the **computational constraints** that you have

Limited Rationality

- **Planning Agent Policy**
  Planning is the process of generating possible sequences of actions, simulating their consequences, picking which is the best and committing to one of these actions
- The (limited rational) agent design problem:
  Find $P^* \rightarrow A$
  - mapping of sequences of percepts to actions
  - maximizes the utility of the resulting sequences of states
  - subject to our computational constraints
- To design an agent, specify PEAS (task environment)

PEAS (Performance, Environment, Actuators, Sensors)

Examples

Automated taxi driver
• P: safe, fast, legal, comfortable trip, maximize profits • E: Roads, other traffic, pedestrians, customers • A: Steering wheel, accelerator, brake, signal, horn • S: cameras, sonar, speedometer, GPS, odometer, engine sensors, keyboard
Medical diagnosis system Agent
• P: Healthy patient, minimize costs, lawsuits • E: Patient, hospital, staff • A: Screen display (questions, tests, diagnoses, treatments, referrals) • S: keyboard (entry of symptoms, findings, patient's answers)
Part-picking robot Agent
• P: percentage of parts in correct bins • E: conveyor belt with parts, bins • A: jointed arm and hand • S: camera, joint angle sensors

Properties of Task Environment

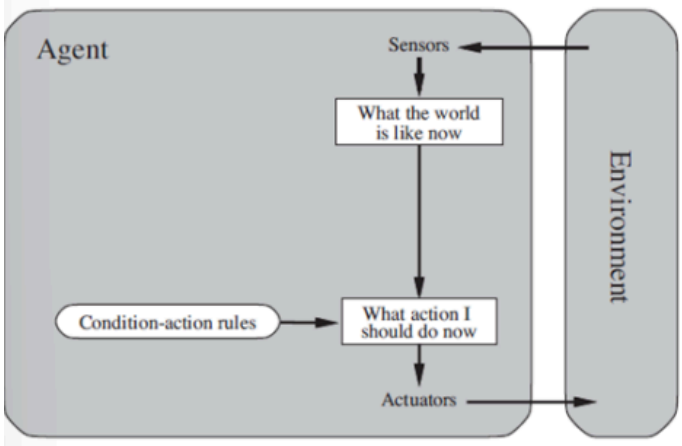
- **Fully observable** (vs. partially observable)
  An agent's sensors give it access to the complete state of the environment at each point in time
- **Deterministic** (vs. stochastic)
  The next state of the environment is completely determined by the current state and the action executed by the agent
- **Episodic** (vs. sequential)
  The agent's experiment is divided into atomic "episodes", each episode consists of the agent perceiving and then performing a single action, and the choice of action in each episode depends only on the episode itself
- **Static** (vs. dynamic)
  The environment is unchanged while an agent is deliberating.
  The environment is **semidynamic** if the environment itself doesn't change with the passage of time but the agent's performance score does
- **Discrete** (vs. continuous)
  A limited number of distinct, clearly defined percepts and actions
- **Single agent** (vs. multiagent)
  An agent operating by itself in an environment
- [not strictly a property of the environment] **Known** (vs. Unknown)
  Refers not to the environment itself but to the agent's state of knowledge about the "laws of physics" of the environment

Agent Programs

Table-lookup Agent

- Table-lookup → maps sequence of percepts to actions
- Drawbacks:
  - huge table
  - take a long time to build table
  - no autonomy
  - even with learning, need a long time to learn the table entries
- Alternatives:
  - reflex agent
    agent respond very flexible to a very broad range of stimuli

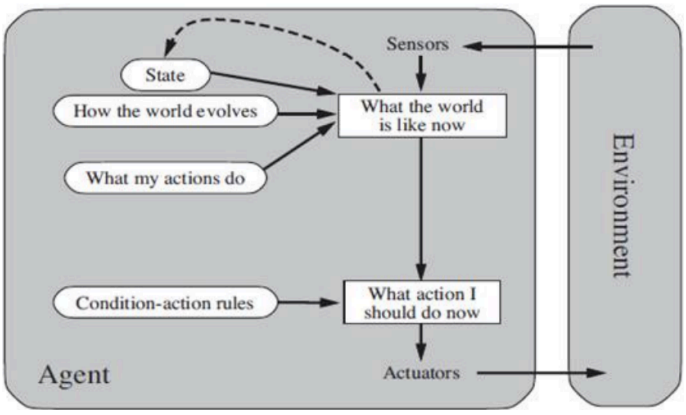
Simple Reflex Agent



function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *rules*, a set of condition–action rules

  state ← INTERPRET-INPUT(*percept*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

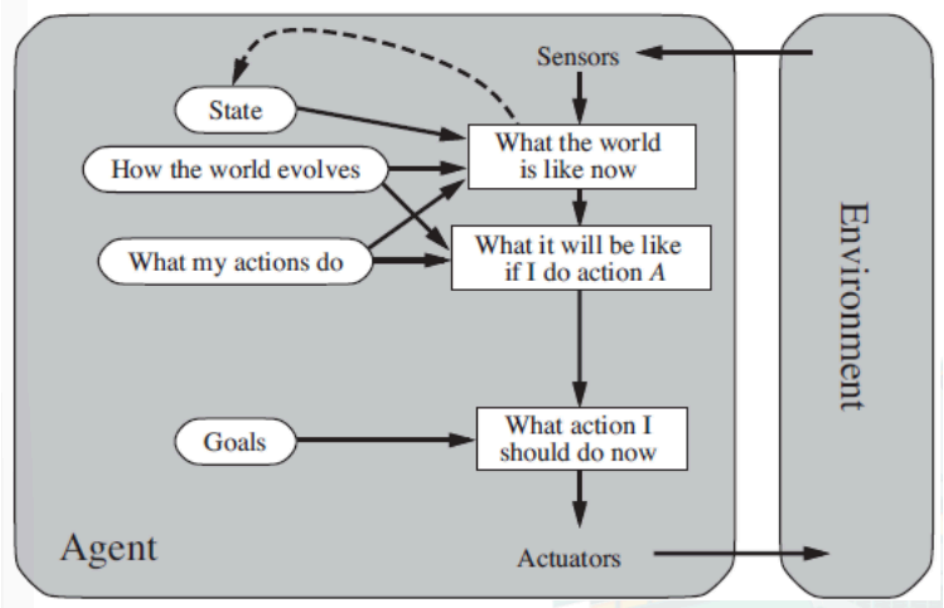
Model-based Reflex Agent



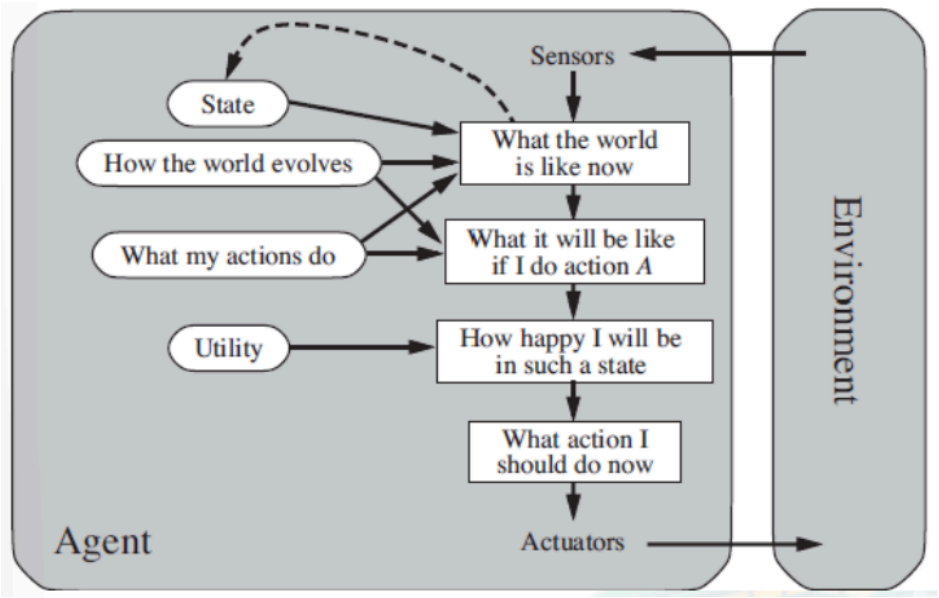
function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *state*, the agent's current conception of the world state
    transition_model, a description of how the next state depends on the current state and action
    sensor_model, a description of how the current world state is reflected in the agent's percepts
    rules, a set of condition–action rules
    action, the most recent action, initially none

  state ← UPDATE-STATE(*state*, *action*, *percept*, *transition_model*, *sensor_model*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

Goal-based Agent



Utility-based Agent

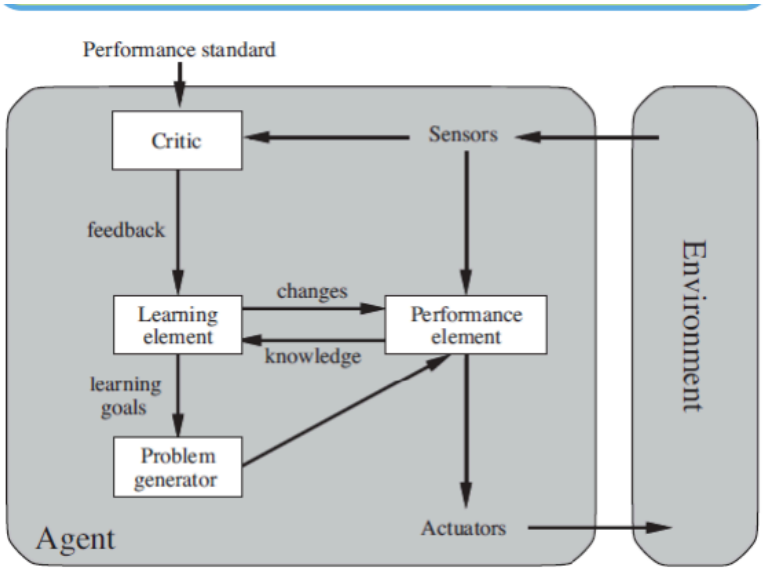


Learning-based Agent

Critic tells the learning element how well the agent is doing with respect to a fixed performance standard

The **learning element** (responsible for making improvement) uses feedback from the critic on how the agent is doing and determines how the performance element should be modified to do better in the future

Problem Generator is responsible for suggesting actions that will lead to new and informative experiences



Performance element responsible for element selecting external actions. The performance element is what we have previously considered to be the entire agent: it takes in percepts and decides on actions

AI Agents vs Agentic AI

Aspek	AI Agent	Agentic AI	Generative Agent
Definisi	Sistem/software otonom yang menjalankan tugas tertentu	Sistem yang memungkinkan beberapa AI berkolaborasi untuk mencapai tujuan kompleks	AI yang berfokus pada menghasilkan konten atau respons baru
Kemampuan utama	Task execution	Autonomous goal setting	Content generation
Tingkat otonomi	Tinggi	Lebih tinggi	Bergantung pada instruksi pengguna; tidak berfokus pada otonomi
Planning horizon	Single-step	Multi-step	N/A
Kompleksitas tugas	Tugas spesifik dan relatif terdefinisi	Tugas kompleks dan multi-step yang membutuhkan koordinasi	Tidak menjadi karakteristik utama

Kolaborasi	independen	Dapat melibatkan kolaborasi multi-agent dan pertukaran informasi	Tidak menjadi karakteristik utama
Learning mechanism	Rule-based atau supervised learning	Reinforcement learning/meta-learning	Large-scale pre-training
Gaya interaksi	Reaktif	Proaktif	Kreatif
Fokus evaluasi	Accuracy dan latency	Engagement dan adaptability	Coherence dan diversity
Contoh aplikasi	Chatbot layanan, virtual assistant, automated workflow	Supply-chain management, business process optimization, virtual project manager	ChatGPT, image generator, code generator, dan sistem generative lainnya

Leveling of Agent

Problem Solving Agent

Knowledge Based Agent

Learning Agent

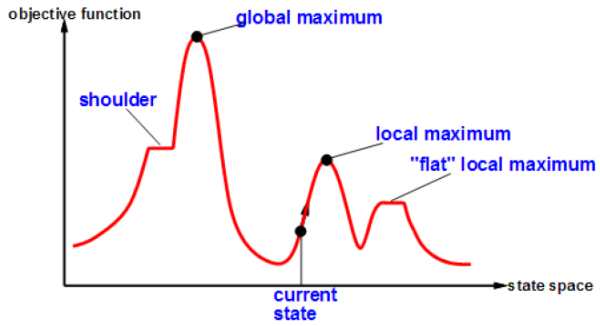
Beyond Classical Search

- Classical vs. Local Search
  - State-space Landscape
- Local Search
  - State, Successor, Neighbor
    - State
    - Successor
    - Neighbor
  - Hill-climbing Search
    - Varian HCS
      - Steepest-Ascent Hill Climbing
      - Hill-Climbing with Sideways Move
      - Random-Restart Hill Climbing
      - Stochastic Hill-Climbing
  - Simulated Annealing
  - Genetic Algorithm

Classical vs. Local Search

	Classical Search	**Local Search**
Representasi state	Dapat berupa **partial** ataupun **complete configuration**	Selalu **complete configuration**
Initial state	**State awal** yang diberikan (dapat berupa konfigurasi parsial)	State yang dibangkitkan secara **acak**
Eksplorasi	Mengeksplorasi ruang state secara sistematis	Bergerak dari satu state ke neighbor
Aksi	Menghasilkan successor berdasarkan operator	Memindahkan current state ke neighbor
Path	**Penting**, karena path to goal merupakan bagian dari solusi	**Tidak penting**, karena solusi berupa final configuration/state
Path cost	Digunakan untuk menentukan kualitas solusi	Tidak digunakan
Goal test	Memeriksa apakah state memenuhi kondisi tujuan/goal	Tidak menggunakan goal test tradisional, namun mencari **test dengan nilai optimum**
Fungsi evaluasi	Digunakan untuk menentukan urutan eksplorasi	**State value** berdasarkan objective/heuristic function
Solusi	Path atau sequence of actions menuju goal	Final state/configuration
Contoh pada N-queens	Menambahkan queen satu per satu hingga seluruh queen ditempatkan tanpa saling menyerang	Mulai dari konfigurasi lengkap, lalu memindahkan queen ke posisi lain

State-space Landscape



A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum. (Russel & Norvig, 2010)

Local search dapat dipandang sebagai pencarian pada **state-space landscape**. Setiap state memiliki:

- **Location** → ditentukan oleh state
- **Elevation** → nilai objective function atau heuristic cost
- **Global optimum** → state dengan nilai terbaik yang ingin ditemukan

Namun, local search dapat berhenti/terjebak dalam **local optimum**, yaitu state yang lebih baik daripada seluruh neighbor di sekitarnya tetapi belum merupakan global optimum.

Local Search

State, Successor, Neighbor

State

- Local search menggunakan **complete state** (konfigurasi lengkap dari permasalahan)
- Contoh pada 8-queens:
    - Satu state berisi posisi satu queen pada setiap kolom. Setiap state memiliki state value yang menunjukkan seberapa baik konfigurasi tersebut.
    - Fungsi heuristik yang digunakan adalah:
      h = - jumlah pasangan queen yang saling menyerang
    - Semakin sedikit jumlah pasangan queen yang saling menyerang, nilai h semakin besar. Solusi optimal memiliki h = 0.

Successor

- Successor function menghasilkan state baru daru current state berdasarkan suatu action.
- Pada 8-Queens, successor diperoleh dengan **memindahkan satu queen ke baris lain pada kolom yang sama**. Karena terdapat **8 queen** dan masing-masing queen dapat dipindahkan ke **7 posisi** lain, terdapat: **8 * 7 = 56 successor**

Neighbor

- Neighbor merupakan state yang dapat dicapai dengan melakukan satu perpindahan dari current state. Neighbor dapat dipilih dengan beberapa cara, misalnya:
    - **random successor** → pilih acak satu successor
    - **highest-valued successor** → pilih successor dengan nilai tertinggi

Hill-climbing Search

- Hill-climbing search adalah local search yang mulai dari sebuah state awal, kemudian bergerak menuju state dengan nilai yang lebih baik
- Algoritma HCS akan:
    - mulai dari **randomly generated initial state**
    - evaluasi neighbor dari current state
    - bergerak ke arah **increasing objective value** atau **decreasing cost**
    - berhenti ketika tidak terdapat neighbor yang memiliki nilai lebih tinggi
- Problem with HCS:
    - **local maximum**: tidak ada neighbor yang lebih baik, tetapi state bukan global maximum
    - **flat local maximum/plateau**: area di mana beberapa state memiliki nilai yang sama sehingga tidak terdapat arah peningkatan yang jelas
    - **shoulder**: area datar yang sebenarnya masih dapat mengarah menuju global maximum

Varian HCS

Varian	**Cara memilih neighbor**	**Karakteristik**
Steepest-ascent HC	Neighbor dengan nilai tertinggi	Memilih langkah terbaik dari seluluh successor
Sideways move	Neighbor yang lebih baik atau	Membantu melewati pleteau

	bernilai sama	
Random-restart HC	Mengulang HC dari initial state acak ketika stuck di local optimum	Mengatasi local optimum dengan restart
Stochastic HC	Successor dipilih secara acak, lalu dievaluasi. Jika lebih baik, diterima.	Lebih sederhana tetapi dapat membutuhkan lebih banyak langkah

Steepest-Ascent Hill Climbing

- Algoritma mengevaluasi seluruh successor dan memilih neighbor dengan nilai paling tinggi
- Pros: memilih langkah terbaik yang tersedia secara lokal
- Cons: dapat terjebak pada local max atau plateau

Hill-Climbing with Sideways Move

- Normal hill-climbing: berhenti ketika tidak ada neighbor yang lebih baik
- Sideways move memungkinkan algoritma berpindah ke neighbor dengan **nilai yang sama** ⇒ membantu algoritma melewati area flat/plateau
- Namun, pencarian menjadi lebih lambat

Random-Restart Hill Climbing

- Jika algoritma gagal karena terjebak di local optimum, algoritma dapat **memulai kembali dari initial state baru dan acak**
- Proses akan diulang sampai solusi ditemukan
- Jika probabilitas keberhasilan adalah ***p***, maka jumlah restart yang diharapkan adalah sekitar ***1/p***.

Stochastic Hill-Climbing

- Stochastic hill-climbing **memilih successor secara acak**.
    - pilih satu successor secara acak
    - jika successor tersebut lebih baik drpd current state, pindah ke successor
    - jika tidak, jangan lakukan perpindahan
    - ulangi hingga mencapai batas *nmax* iterasi
- Dapat membutuhkan lebih banyak langkah

Simulated Annealing

- Simulated annealing memungkinkan algoritma melakukan **downhill move**, yaitu perpindahan ke state yang lebih buruk. Hal ini dilakukan agar algoritma dapat **keluar dari local optimum**.
- Allow some bad moves → gradually decrease their frequency
- Pada simulated annealing, neighbor yang lebih buruk masih dapat diterima dengan probabilitas tertentu. Sedangkan pada stochastic HC, neighbor yang lebih buruk langsung ditolak.
- **Temperature (T)**

Untuk:
$\Delta E = value(neighbor) - value(current)$
- Jika $\Delta E > 0 \rightarrow$ neighbor lebih baik, **pindah**
- Jika $\Delta E < 0 \rightarrow$ neighbor lebih buruk, **pindah berdasarkan probability**

Probabilitas perpindahan ke state yang lebih buruk:
$P = e^{\Delta E/T}$

- Ketika nilai **T tinggi**, kemungkinan menerima bad move lebih besar sehingga algoritma lebih banyak melakukan random exploration.
- Ketika **T mendekati 0**, kemungkinan menerima bad move semakin kecil sehingga perilakunya semakin menyerupai stochastic hill-climbing

Genetic Algorithm

- Genetic algorithm tidak memproses state secara independen. Informasi dari state yang baik dapat **digabungkan** untuk menghasilkan state baru
- **State/Individual**
  - Individual direpresentasikan sebagai string dari suatu finite alphabet
- **Fitness function**
  - Setiap individual memiliki **fitness value** yang menunjukkan seberapa baik individual tersebut. Higher fitness = better state
- **Proses GA**:
  1. **Selection**
     a. Memilih parent dari populasi berdasarkan fitness
     b. Individual dengan fitness lebih tinggi memiliki **probabilitas lebih besar untuk dipilih**

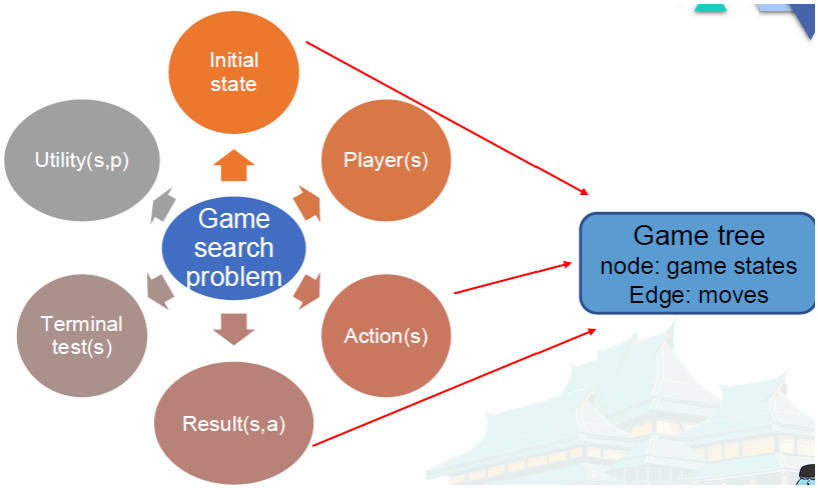
c. **Misal**: terdapat fitness value 24, 23, 20, 11. Probabilitas pemilihannya dihitung berdasarkan proporsi fitness masing-masing. Fitness 24 memiliki probabilitas sekitar 31%, sedangkan fitness 11 sekitar 14%

2. **Crossover**
    a. Dua parent yang telah dipilih **digabungkan** untuk menghasilkan offspring
    b. Crossover dilakukan dengan menentukan **crossover point** secara random, kemudian bagian dari kedua parent dikombinasikan
3. **Mutation**
    a. Setelah crossover, satu atau beberapa bagian offspring dapat diubah pada **mutation point** yang dipilih secara random
    b. Mutation berfungsi menghasilkan variasi baru pada populasi sehingga pencarian tidak hanya bergantung pada kombinasi dari solusi yang sudah ada

Adversarial Search

- Pada non-adversarial search, algoritma mencari tindakan terbaik untuk mencapai tujuan
- Adversarial search adalah metode pencarian untuk menentukan langkah terbaik dalam suatu permainan dengan mempertimbangkan bahwa lawan juga akan memilih langkah terbaik untuk mengalahkan kita

Game Tree



Node: kondisi permainan
Edge: langkah
Root: kondisi awal
Leaf/terminal state: permainan sudah selesai

Minimax Algorithm

- MAX → ingin memaksimalkan nilai → kita
- MIN → ingin meminimalkan nilai → lawan
- Properti minimax:
    - complete? → ya, jika tree finite
    - optimal? → ya, terhadap lawan yang optimal
    - time complexity → $O(b^m)$
    - space complexity → $O(bm)$ dengan depth-first exploration

      b = branching factor
      m = maximum depth

α-β Pruning

- Optimasi minimax dengan tujuan untuk menghindari pemeriksaan cabang yang sudah pasti tidak akan memengaruhi keputusan akhir
- α = best MAX value so far
- β = best MIN value so far
- Pruning terhadap cabang tertentu dapat dilakukan jika **α >= β**

Constraint Satisfaction Problem (CSP)

What is CSP
Jenis Constraint
Constraint Graph

What is CSP

- CSP adalah masalah pencarian solusi yang didefinisikan menggunakan:
    - **Variables**: sekumpulan variabel X_i
    - **Domains**: sekumpulan nilai D_i yang mungkin diberikan kepada setiap variabel
    - **Constraints**: aturan yang menentukan kombinasi nilai yang diperbolehkan untuk variabel-variabel tersebut
- CSP mencari assignment nilai untuk semua variabel sehingga seluruh constraint terpenuhi
- Pada CSP, struktur state dinyatakan secara eksplisit melalui **variables dan domains**, sedangkan goal test berupa **constraints**
- Contoh pada masalah pewarnaan peta:
    - Variables: WA, NT, Q, NSW, V, SA, T
    - Domain setiap var: D_i = {red, green, blue}
    - Constraint: daerah yang bersebelahan harus memiliki warna berbeda
    - Salah satu solusi:
        - WA = red
        - NT = green
        - Q = red
        - NSW = green
        - V = red
        - SA = blue
        - T = green
    - Sebuah solusi bersifat **complete dan consistent assignment** (semua variabel bernilai dan tidak ada constraint yang dilanggar)

Jenis Constraint

- Constraint dapat dibedakan berdasarkan jumlah variabel yang terlibat
- **Unary constraint**
    - melibatkan 1 variabel, contohnya: SA ≠ green
- **Binary constraint**
    - melibatkan 2 variabel, contohnya: SA ≠ WA
- **Higher-order constraint**
    - melibatkan >=3 variabel, contohnya adalah constraint pada masalah cryptarithmetic

Constraint Graph

- Constraint graph digunakan untuk merepresentasikan hubungan antarvariabel dalam binary CSP

- Pada constraint graph:
    - node = variabel
    - arc/edge = constraint antara dua variabel
- Constraint graph membantu kita melihat variabel mana yang saling memengaruhi
- Untuk constraint yang melibatkan lebih dari 2 variabel, digunakan **constraint hypergraph**
    - node = variabel,
    - kotak (hypernode) = n-ary constraint

Inference in CSP

- Inference = untuk mengurangi kemungkinan solusi yang harus diperiksa
- Inference dilakukan melalui **constraint propagation**
- Misalnya:
    - SA = {red, blue green}; WA = {red, blue}
    - Terdapat constraint: SA ≠ WA
    - Jika diketahui SA = red
    - Maka: WA = {blue} untuk memenuhi constraint di atas
- Constraint propagation dapat dilakukan:
    - **sebelum search** sebagai preprocessing, atau
    - **bersamaan** dengan **search** untuk early failure detection

Node consistency

- Sebuah variabel dikatakan **node-consistent** jika **semua nilai dalam domainnya memenuhi unary constraint**
- sebuah network disebut node-consistent jika setiap variabel dalamnya node-consistent

Arc consistency

- Sebuah variabel dikatakan **arc-consistent** jika setiap nilai pada domain variabel tersebut memiliki setidaknya satu nilai pada domain variabel lain yang memenuhi binary constraint
- Arc X → Y dikatakan consisten jika **untuk setiap nilai x pada X terdapat setidaknya satu nilai y pada Y yang diperbolehkan constraint**

Path consistency

- Path consistency melihat konsistensi suatu pasangan variabel terhadap variabel ketiga

- Pasangan $\{X_i, X_j\}$ dikatakan path-consistent terhadap X_m jika setiap assignment yang konsisten untuk X_i dan X_j memiliki setidaknya satu assignment pada X_m yang juga memenuhi constraint dengan keduanya.

K-consistency

CSP disebut k-consistent jika untuk setiap assignment konsisten dari k - 1 variabel, selalu terdapat nilai yang dapat diberikan kepada variabel ke-k sehingga assignment tetap konsisten

Backtracking Search untuk CSP

- Inference saja tidak selalu cukup untuk menyelesaikan CSP. Pada titik tertentu kita perlu melakukan search.
- **Backtracking search**: DFS yang memberikan assignment kepada satu variabel pada setiap langkah
- Why DFS? karena setiap solusi lengkap berada pada depth n, dimana n adalah jumlah variabel
- Selain itu, **urutan tidak penting.**

Improving Backtracking Search

→ backtracking dasar bisa menjadi sangat mahal

Variable Ordering

→ which variable should be picked first?

Static Order

- Variabel selalu dipilih berdasarkan urutan yang telah ditentukan
- Contoh: WA, NT, SA, Q, NSW, V, T

Most Constrained Variable (MRV)

- Pilih variable yang memiliki **paling sedikit legal values**
- AKA **Minimum Remaining Values (MRV), Fail-first heuristic**
- Tujuan: menemukan kegagalan secepat mungkin
- Contoh: SA = {blue}; Q = {blue,red} maka SA dipilih terlebih dahulu karena hanya memiliki satu kemungkinan

Most Constraining Variable

- Jika ada beberapa variabel yang sama-sama paling constrained, gunakan **degree heuristic** sebagai tie-breaker
- Pilih variabel yang memiliki **constraint terbanyak terhadap remaining variables**

Value Ordering

→ value mana yang dicoba terlebih dahulu?

- Heuristik **Least Constraining Value**: pilih value yang mengeliminasi paling sedikit kemungkinan value pada variabel lain ⇒ pilih value yang memberikan kebebasan terbesar ke variabel lain
- Contoh: X = {red, blue} Jika X = red, ia akan menghilangkan 5 kemungkinan. namun jika X = blue, ia hanya menghilangkan 1 kemungkinan. Maka pilih X = blue terlebih dahulu.

Interleaving Search and Inference

- Inference can be done before searching
- Interleaving search and inference → detect failure early
- Ada dua pendekatan:
    - **Forward checking**: establishes arc consistency for binary constraint
    - **Constraint propagation**: Maintaining Arc Consistency (MAC)

Forward Checking

- Ide: menyimpan kemungkinan nilai yang masih legal untuk setiap unassigned variable dan menghentikan search jika salah satu variabel tidak lagi memiliki nilai legal
- Misal:
    - SA = {red, blue}
      WA = {red, blue}
    - Lalu dipilih SA = red
    - Karena SA ≠ WA maka red dihapus dari domain WA: WA = {blue}
    - Jika setelah propagasi ternyata WA = {} maka kita langsung tahu bahwa assignment sebelumnya menghasilkan kegagalan dan tidak perlu melanjutkan search lebih dalam
- Assign var → update neighbor's domain → any empty domain → if yes, backtrack; if no, continue search

Maintaining Arc Consistency (MAC)

- Forward checking menyebarkan informasi dari assigned variables ke unassigned variables. Sementara MAC terus melakukan propagation sehingga **arc consistency tetap terjaga**.
- $X \rightarrow Y$
  Jika suatu value di X dihapus, maka **tetangga X harus diperiksa kembali** karena pengurangan domain X dapat membuat value tertentu di tetangganya menjadi tidak valid.

Knowledge Based System

- What is KBS
- Why KBS?
- Knowledge Based Agent
- Knowledge Representation
  - Teknik KR
    - Logika Proposisional dan Predikat
    - Semantic Networks/Ontology
    - Frames
    - Production Rules
    - Case-based Reasoning
    - Bayes Network
- Arsitektur KBS
  - Interactive vs Embedded KBS
  - Arsitektur Umum KBS
    - Examples
- Akuisisi dan Rekayasa Pengetahuan
  - Direct vs Indirect Knowledge Acquisition
  - Pihak yang terlibat dalam pengembangan KBS
  - Tugas dalam Knowledge Elicitation
  - Tahapan Akuisisi Pengetahuan
  - Teknik Akuisisi Pengetahuan
    - Manual
    - Otomatis

What is KBS

- Knowledge Based System (KBS) adalah sistem yang menggunakan knowledge untuk menyelesaikan suatu masalah
- KBS berusaha merekonstruksi **expertise dan reasoning capabilities** dari spesialis/pakar dalam domain tertentu
- **KBS = Problem-solving method + Knowledge + Data**
    - **Knowledge** → pengetahuan mengenai domain masalah
    - **Data** → fakta/informasi yang tersedia
    - **Problem-solving method** → algoritma/metode yang menentukan bagaimana knowledge domain digunakan untuk menyelesaikan masalah
- Perbedaan KBS & conventional program:

Conventional program	**Knowledge based system**
algorithm + data	problem-solving method + knowledge + data
programmer menentukan aksi dan urutan eksekusi	expert menentukan knowledge/aksi, interpreter menentukan urutan eksekusi
cocok untuk masalah well-structured	dapat menangani masalah yang lebih ill-structured

- KBS ≠ expert system
    - Expert system → salah satu bentuk aplikasi dari KBS.

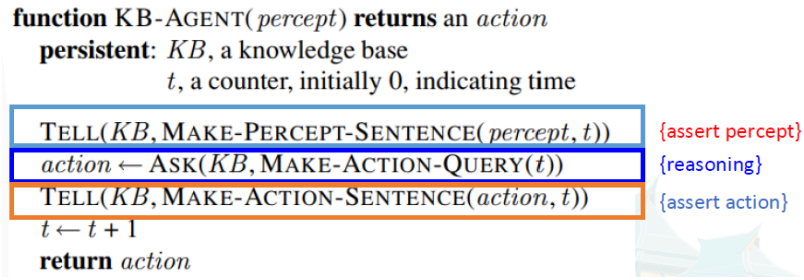
Why KBS?

- untuk melakukan **logical reasoning** → pendekatan **thinking rationally**
    - Dalam knowledge-based agent: pada setiap langkah, agent menarik kesimpulan dari informasi yang tersedia
    - Salah satu sifat fundamental logical reasoning: "Jika informasi yang tersedia benar, maka kesimpulan yang dihasilkan dijamin benar"
- KBS cocok untuk digunakan terutama ketika:
    - domain memiliki knowledge yang dapat direpresentasikan secara eksplisit
    - proses penyelesaian membutuhkan reasoning
    - solusi tidak selalu dapat ditulis sebagai algoritma sederhana
    - knowledge domain dapat berubah atau dikembangkan secara terpisah dari mekanisme reasoning

Knowledge Based Agent

- Agent yang menggunakan knowledge untuk mengambil keputusan

- Alur: **Percept → Knowledge base → Reasoning → Action**
    - Agent menerima informasi dari lingkungan , memasukkan informasi tersebut ke knowledge base, melakukan reasoning lalu menentukan aksi
- Contoh: Wumpus world
    - Misal, agent berada di [1,1] dan mendapatkan
      Percept = [None, None, None, None, None]
    - Dari tidak adanya stench, agent dapat menyimpulkan tidak ada Wumpus di lokasi tetangga tertentu. Dari tidak adanya breeze, agent dapat menyimpulkan tidak ada pit di lokasi tetangga tertentu. Agent kemudian memilih action berdasarkan kesimpulan tersebut
- Generic KBA:



- Pengembangan KBA:
    - Mulai dari Knowledge Base kosong → perancang belum memiliki gambaran solusi
    - Perancang men-TELL kalimat satu per satu → perancang tidak bisa mengantisipasi semua situasi yang mungkin
    - Berhenti saat agen tahu cara beroperasi di lingkungannya. Namun siklus kembali ke tahap TELL karena perancang tidak bisa mengantisipasi semua perubahan dari waktu ke waktu

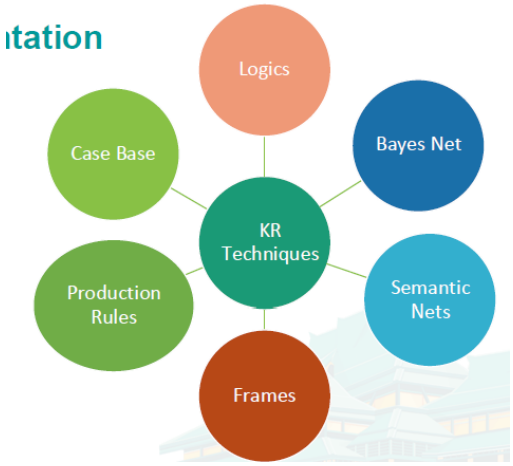
Knowledge Representation

- Knowledge representation (KR) adalah bahasa untuk menyatakan pengetahuan/informasi. Bahasa ini berupa aturan sintaks dan semantik yang memungkinkan kita mendeskripsikan sesuatu, ditambah cara memanipulasi ekspresi dalam bahasa itu
- Bahasa KR memiliki dua sisi yang harus ada bersamaan:
    - **Syntax**: apa yang boleh ditulis, yaitu ekspresi yang sah dalam bahasa tersebut
    - **Semantic**: makna dari ekspresi tersebut
- Syarat representasi pengetahuan yang baik:
    - tidak ada kontradiksi

    - setiap simbol harus unik
    - mampu menjelaskan objek, relasi, dan atribut tertentu
    - efisien dimanipulasi di sistem komputer
- Memilih representasi pengetahuan yang tepat:

Kriteria	**Pilihan yang cocok**
Suitable for **problem domain**	• Decision tree for classification • Skeletal construction for construction • Rule for all problem domain
Suitable for the **tasks** (inference)	• Decision tree including interview process • Probability model for decision with uncertainty
Suitable for **users** (man or machine)	• Semantic network for user • Rule for machine

Teknik KR



Logika Proposisional dan Predikat

- Merepresentasikan pengetahuan dengan **aturan IF-THEN, hubungan antar objek, dan kuantifikasi objek**

- Contoh:
    - Smart building control system (sensor mendeteksi tidak ada orang, lampu dimatikan otomatis, memakai logika proposisional)
    - Automated theorem proving dengan logika predikat, misalnya program Prolog yang menjawab pertanyaan logis dari fakta dan aturan

Semantic Networks/Ontology

- Pengetahuan berbentuk graf:
    - node adalah konsep/objek
    - edge adalah hubungan antar konsep
- Contoh:
    - google knowledge graph (pencarian semantic)
    - paypal fraud detection (relasi akun, lokasi, jenis transaksi, sejarah pembayaran untuk menemukan aktivitas mencurigakan)
    - penelitian medis (hubungan baru antara penyakit, gen, dan obat)

Frames

- Struktur data berupa **slot** (tempat menyimpan informasi) dan **filler** (nilai yang mengisi slot)
- Contoh:
    - chatbot dan asisten virtual (memahami konteks pertanyaan)
    - sistem navigasi robot (pengetahuan lingkungan dan tindakan robot)

Production Rules

- Pengetahuan berbentuk **aturan kondisi-aksi** (**if-then rules**)
- Contoh:
    - MYCIN (sistem pakar medis tahun 1970an untuk diagnosis infeksi bakteri, misalnya if pasien memiliki gejala X dan hasil tes Y, then infeksi bakteri Z terindikasi)
    - Snort (sistem deteksi intrusi open-source, aturan dapat disesuaikan untuk mendeteksi DDoS, brute force, malware)

Case-based Reasoning

- Solusi dari kasus sebelumnya dipakai ulang untuk masalah baru yang serupa
- Bergantung pada basis data kasus berisi solusi/keputusan masa lalu
- Contoh:

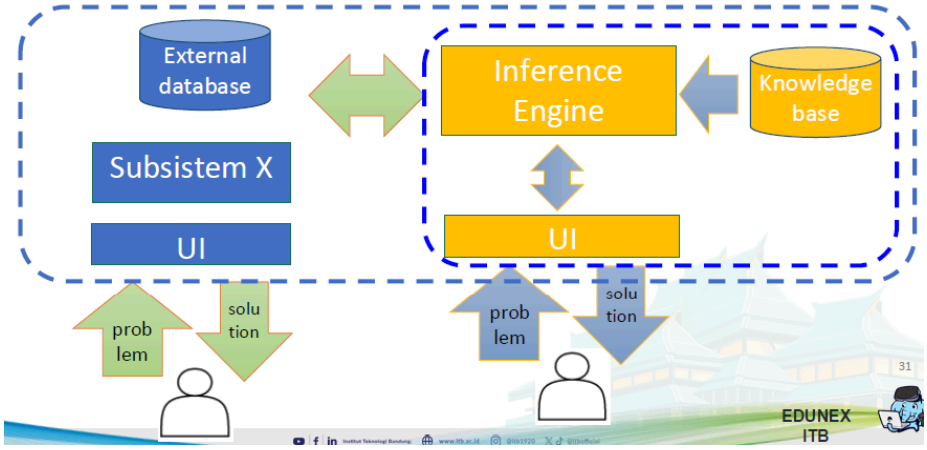
- Sistem radiologi (mendiagnosis gambar medis berdasarkan kasus yang sudah didiagnosis)
- HelpDesk (merujuk kasus teknis yang sudah terpecahkan untuk menyarankan solusi)

Bayes Network

- Untuk **reasoning under uncertainty**
- Ketidakpastian direpresentasikan tidak hanya lewat **disjungsi (logika)**, tetapi juga lewat **kemungkinan (probabilitas)**

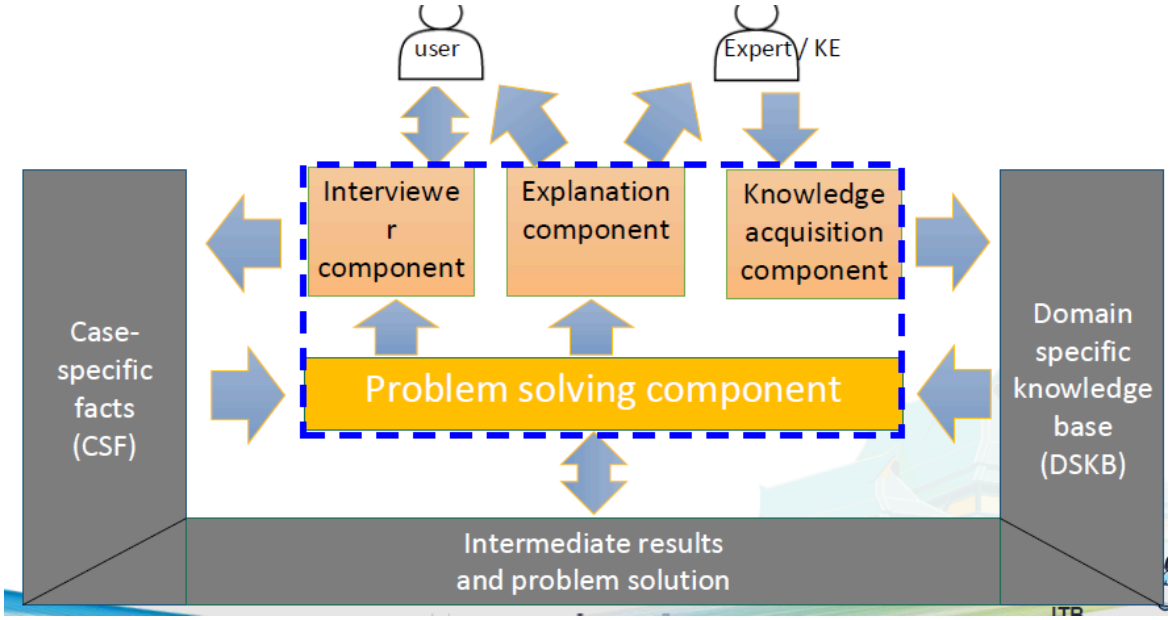
Arsitektur KBS

Interactive vs Embedded KBS



Jenis	Ciri	Komponen di diagram
Interactive	Pengguna berinteraksi langsung dengan KBS lewat UI	UI, inference engine, knowledge base
Embedded	KBS tertanam di sistem lain dan bertukar data dengan subsistem/database	Subsistem X, database external, UI milik subsistem, ditambah inference engine dan knowledge base

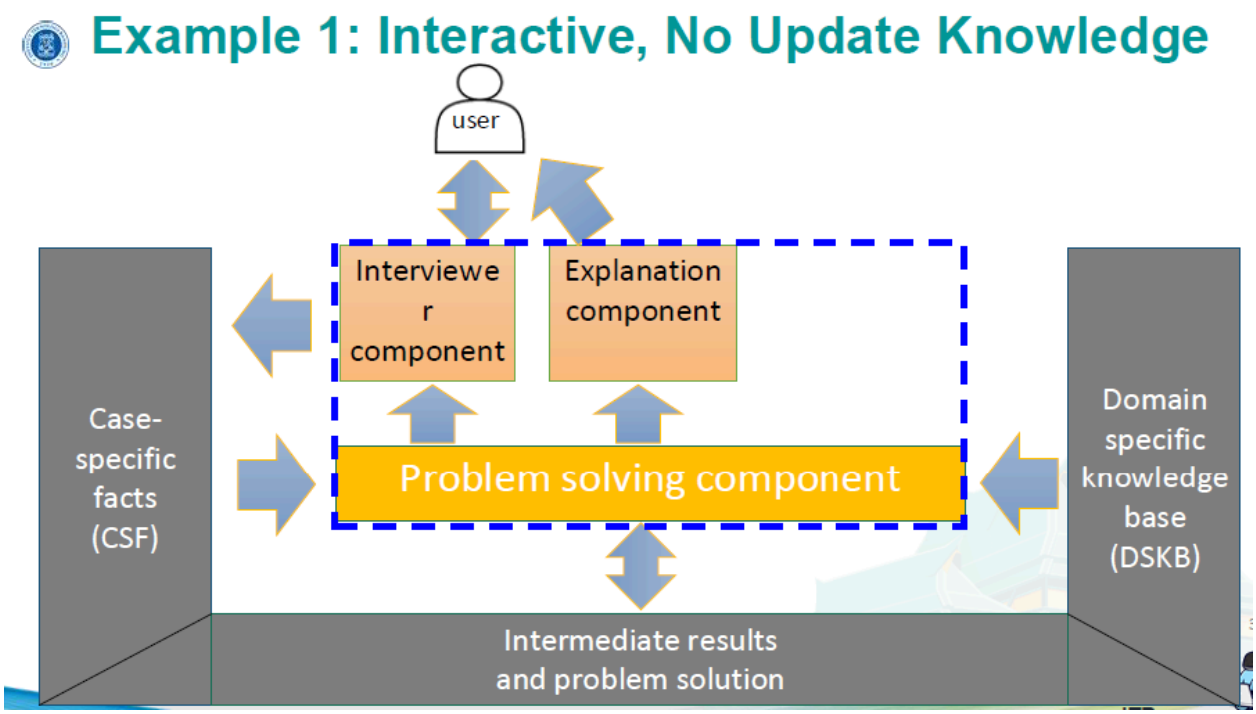
Arsitektur Umum KBS

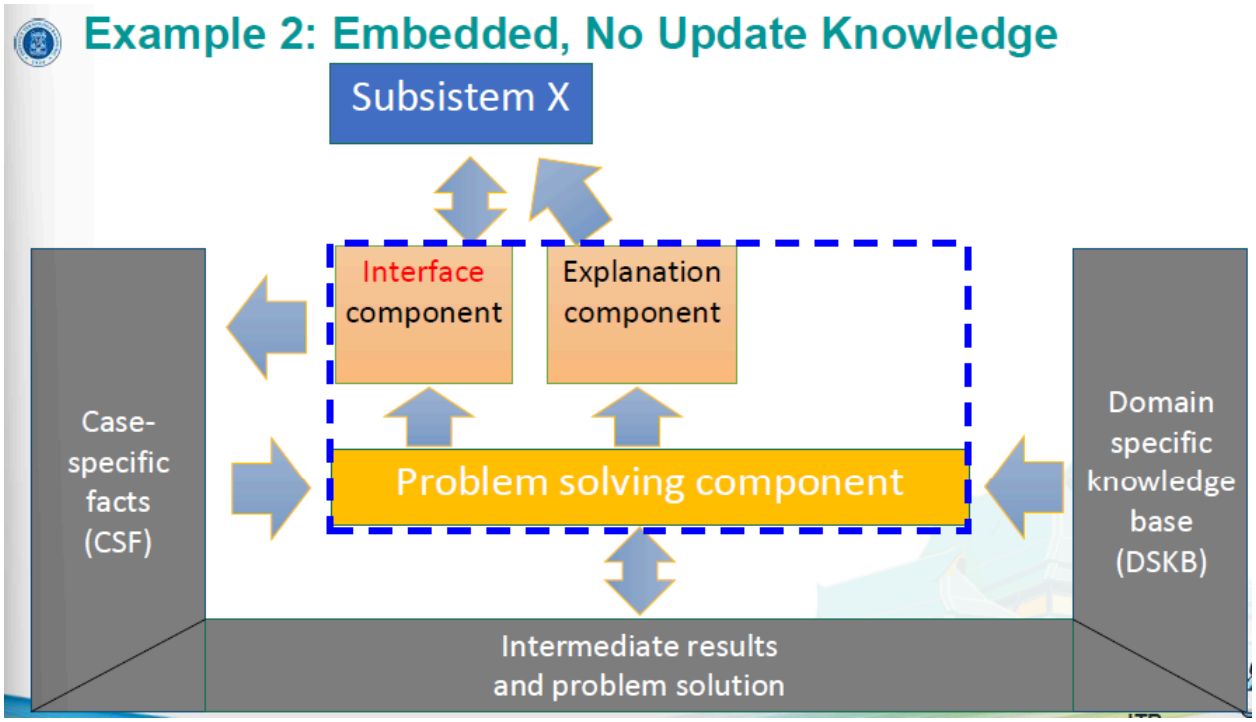


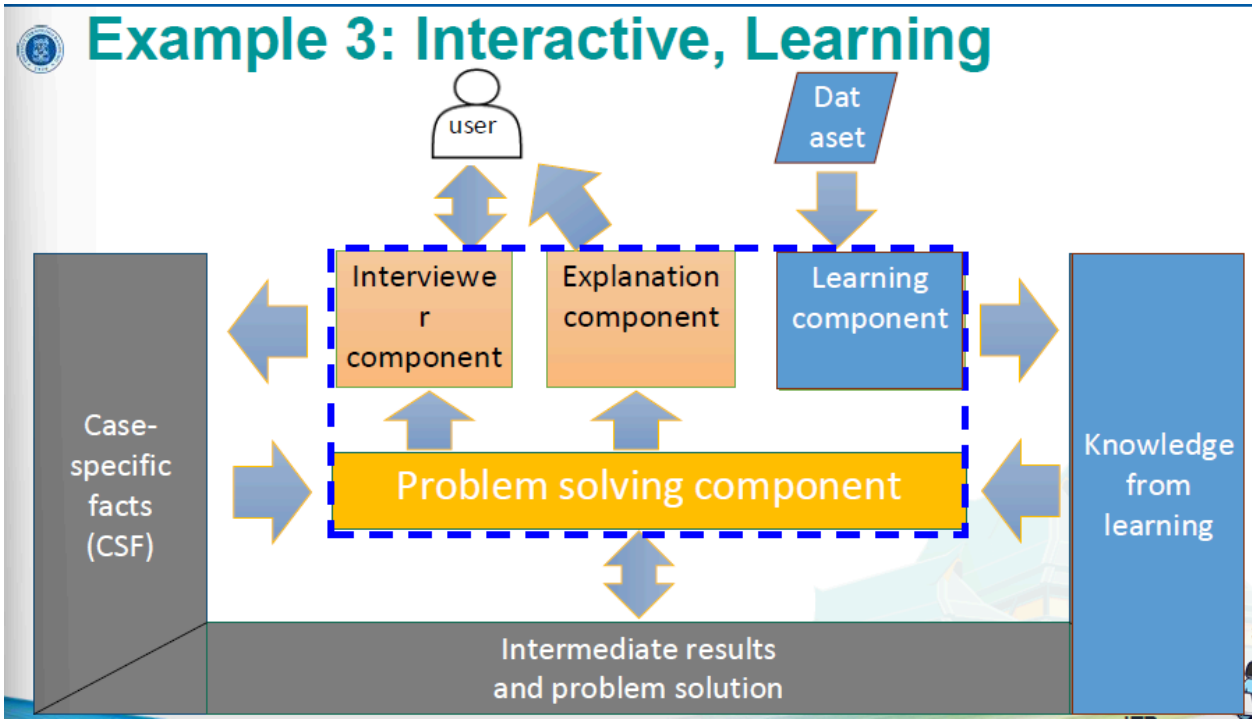
Komponen dan fungsinya:

- **Problem solving component**: inti sistem, menjalankan penalaran
- **Domain-specific knowledge base (DSKB)**: pengetahuan khusus domain, **berubah lambat** dan disimpan terpisah
- **Case-specific facts (CSF)**: fakta untuk kasus yang sedang diselesaikan
- **Intermediate results and problem solution**: hasil sementara dan solusi akhir
- **Interviewer component**: berinteraksi dua arah dengan user untuk mengumpulkan fakta kasus, lalu mengisi CSF
- **Explanation component**: menjelaskan hasil/penalaran ke user
- **Knowledge acquisition component**: dipakai **expert/knowledge engineer (KE)** untuk mengisi DSKB

Examples


Example 1: Interactive, No Update Knowledge
user
Interviewe r component
Explanation component
Problem solving component
Case-specific facts (CSF)
Domain specific knowledge base (DSKB)
Intermediate results and problem solution



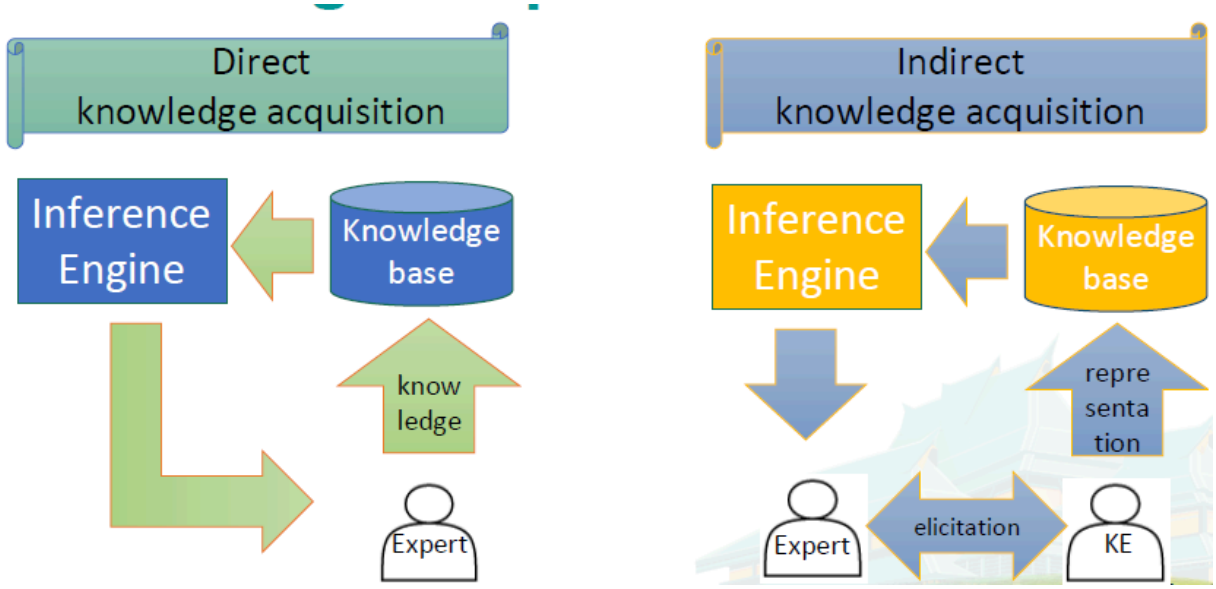


Akuisisi dan Rekayasa Pengetahuan

- **Rekayasa pengetahuan (knowledge engineering)**: memperoleh pengetahuan suatu domain dari satu atau lebih **sumber non-elektronik** (misalnya pakar), lalu mengubahnya ke bentuk yang bisa dipakai komputer untuk memecahkan persoalan yang biasanya hanya bisa dipecahkan pakar domain tersebut

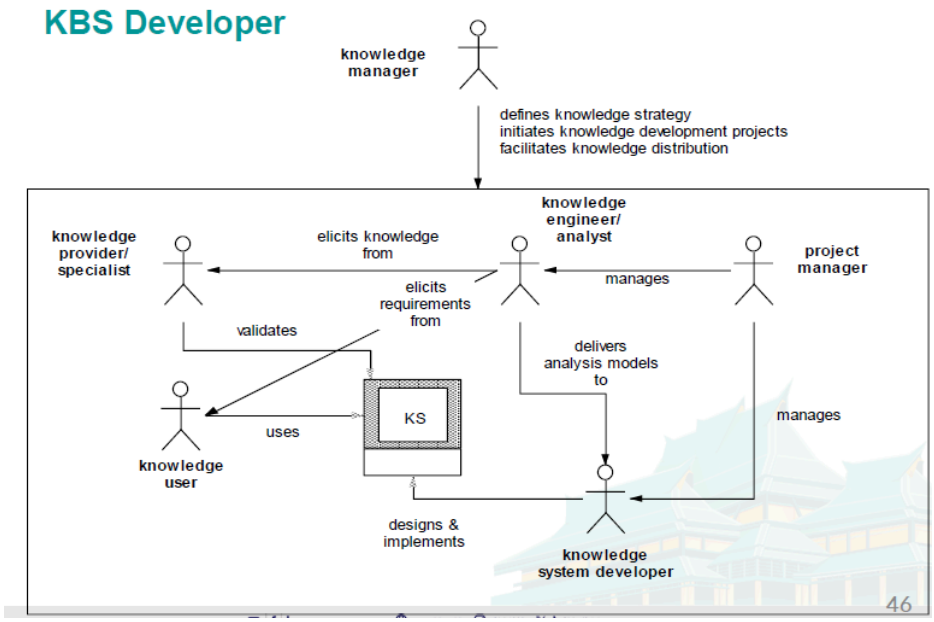
- **Akuisisi pengetahuan (KA) = knowledge elicitation + knowledge representation**
    - **Elicitation:** proses mengekstrak pengetahuan domain dan strategik dari pakar, lewat wawancara antara KE dan pakar. Ini proses siklik (cyclical)
    - **Representation:** proses merepresentasikan pengetahuan hasil ekstraksi ke **bentuk formal**.

Direct vs Indirect Knowledge Acquisition



Jenis	**Cara kerja**
Direct	Expert memasukkan pengetahuan ke knowledge base, tanpa perantara
Indirect	Ada perantara knowledge engineer (KE). Expert dan KE melakukan elicitation, lalu KE membuat representation yang dimasukkan ke knowledge base

Pihak yang terlibat dalam pengembangan KBS



Peran	Tugas
Knowledge manager	Menentukan strategi pengetahuan, memulai proyek pengembangan pengetahuan, memfasilitasi distribusi pengetahuan
Knowledge provider / specialist	Sumber pengetahuan, dan **memvalidasi** sistem pengetahuan (KS)
Knowledge engineer / analyst	**Menggali pengetahuan** dari knowledge provider, **menggali kebutuhan** dari knowledge user, dan **menyerahkan model analisis** ke system developer
Knowledge system developer	**Mendesain dan mengimplementasikan** KS
Project manager	Mengelola knowledge engineer dan system developer
Knowledge user	**Menggunakan** KS

Tugas dalam Knowledge Elicitation

Pada setiap iterasi:

1. Mengumpulkan pengetahuan (i.e. from expert)
2. Menentukan konsep kunci dalam domain masalah

3. Menetapkan hubungan antar konsep dalam domain masalah
4. Memutuskan bagaimana pengetahuan direpresentasikan di KBS
5. Menentukan pengetahuan apa yang perlu dikumpulkan di siklus berikutnya

Tahapan Akuisisi Pengetahuan

Tahap	**Isi**
Identification	Mengidentifikasi karakteristik masalah
Conceptualization	Menemukan konsep-konsep untuk merepresentasikan pengetahuan
Formalization	Mendesain struktur untuk mengorganisasikan pengetahuan
Implementation	Merumuskan pengetahuan menjadi program yang bisa dijalankan
Testing	Memvalidasi pengetahuan

Teknik Akuisisi Pengetahuan

Manual

1. Interview
2. Observasi
3. Intuitif: tukar peran antara KE dan expert

Otomatis

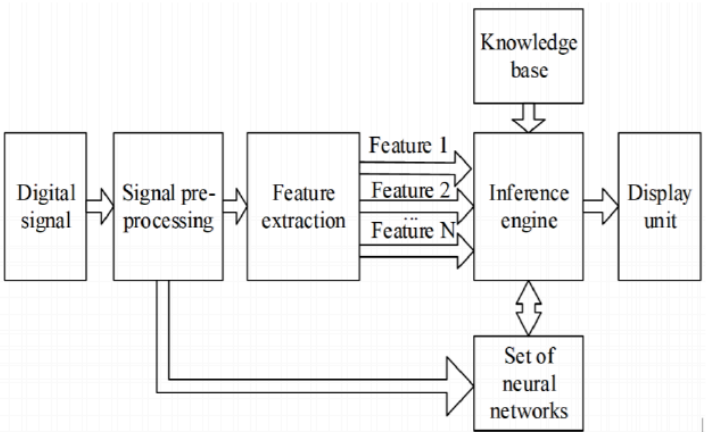
1. Memakai tools untuk memfasilitasi akuisisi
2. tools untuk expert
3. tools machine learning

Rule Based System

What &Why
Fitur RBS
Arsitektur Umum RBS
Pattern Matching & CLIPS
Inference Methods
    Forward Chaining
        Conflict-resolution Strategy
    Backward Chaining
        FINDOUT(GOAL)
        MONITOR(GOAL)

What &Why

- Rule-based System (RBS) adalah KBS yang memakai rule sebagai representasi pengetahuan
- Rule = precondition → action (LHS → RHS)
- Why use RBS:
    - RBS adalah solusi paling sederhana dan paling banyak dipakai di dunia nyata
    - Rule adalah representasi pengetahuan paling sederhana dan paling umum
- Hybrid approach: RBS + ML



- Bentuk rule sebagai implikasi logis:
    - **IF** kondisi benar → disebut **preconditions/premises/LHS**
    - **THEN** jalankan aksi → disebut **actions/conculsion/RHS**
- Dua jenis aksi dalam rule:

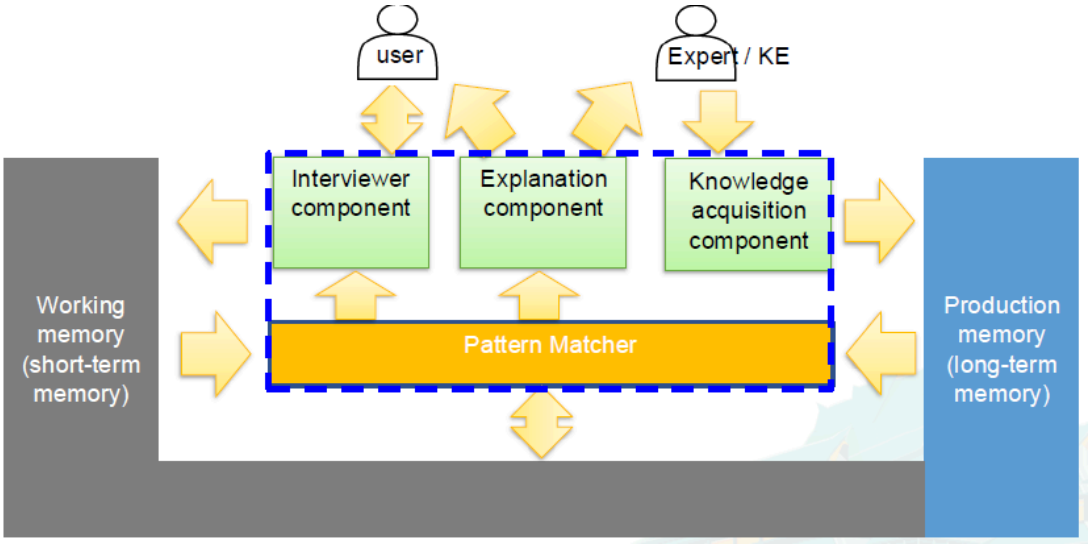
Jenis	Fungsi	Contoh

Implication	**Menurunkan fakta baru**	IF cloudy THEN may_rain; if leher kaku and suhu badan tinggi and kehilangan kesadaran then penyakit meningitis
Instruksi	**Mengubah state sistem**	IF may_rain THEN should_take_umbrella; if clear(box1) and put(box2) then on(box2, box1)

Fitur RBS

- **Modularity**: tiap rule adalah potongan pengetahuan kecil yang relatif independen
- **Incrementability**: rule baru bisa ditambah relatif tanpa menyentuh rule lain
- **Modifiability**: rule lama bisa diubah relatif tanpa menyentuh rule lain
- **Mendukung transparansi sistem**

Arsitektur Umum RBS



Komponen	Keterangan
Working memory (short-term memory)	Menyimpan fakta saat ini
Production memory (long-term memory)	Menyimpan rule
Pattern matcher	Inti sistem: mencocokkan fakta dengan rule
Interviewer component	Interaksi dua arah dengan user
Explanation component	Menjelaskan hasil ke user
Knowledge acquisition component	Dipakai expert/KE untuk mengisi rule

Pattern Matching & CLIPS

- Example:

IF: is-a (x, horse),
is-parent-of(x, *y*),
is-fast(y)
THEN: *x* is valuable

Facts

Comet	is-a	horse
Prancer	is-a	horse
Comet	is-parent-of	Dasher
Comet	is-parent-of	Prancer
Prancer	is	fast
Dasher	is-parent-of	Thunder
Thunder	is	fast
Thunder	is-a	horse
Dasher	is-a	horse

Inference Methods

	Forward chaining (FC)	**Backward chaining (BC)**
Penggerak	Data-driven	Goal-driven
Mencocokkan	LHS rule	RHS rule
Mulai dari	Fakta awal, menurunkan fakta baru sampai syarat berhenti terpenuhi	Pertanyaan/goal, mencari rule yang menyimpulkannya

Forward Chaining

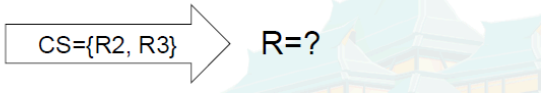
- **Recognize-act cycle**: fakta dan rule masuk ke **pattern matching**, menghasilkan **valid rules** (conflict set, disimpan di agenda), lalu **conflict resolution** memilih satu **selected rule**, rule itu di-fire (mengubah fakta), dan siklus kembali ke fakta
- Pseudo-code:

```
data ← initial facts
repeat
     conflictSet ← determine set of rules whose
                   preconditions are satisfied by data
                   //preselection
     R ← select a rule from conflictSet by conflict-
         resolving strategy
     data ← result of applying action part of R to data
until data satisfied termination condition
```

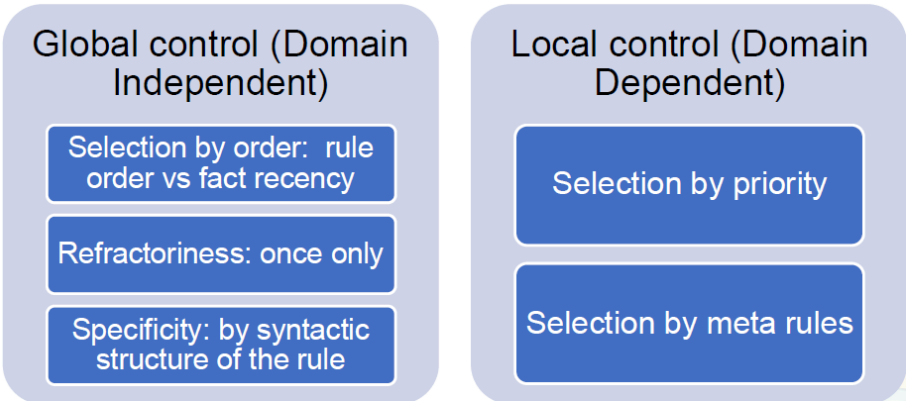
- Example:

Rule-base:
R1: IF (lecturing X) AND (marking-practicals X) THEN ADD (overworked X)
R2: IF (month february) THEN ADD (lecturing alison)
R3: IF (month february) THEN ADD (marking-practicals alison)
R4: IF (overworked X) OR (slept-badly X) THEN ADD (bad-mood X)
R5: IF (bad-mood X) THEN DELETE (happy X)
R6: IF (lecturing X) THEN DELETE (researching X)

Facts:
(month february)
(happy alison)
(researching alison)



Conflict-resolution Strategy



Kelompok	Strategi	Arti
Global control (domain independent)	**Selection by order**	Rule order (urutan rule, FIFO) atau fact recency (fakta terbaru, LIFO)
	Refractoriness	Jangan pilih rule yang baru saja diterapkan dengan nilai variabel yang sama
	Specificity	Pilih rule yang paling spesifik secara struktur sintaks
Local control (domain dependent)	**Selection by priority**	Pilih rule berprioritas tinggi
	Selection by meta rules	Rule tentang rule, misalnya pruning rules yang membuang rule tidak berguna

Backward Chaining

FINDOUT(GOAL)

1. **Jika GOAL dapat disimpulkan** (ada rule yang bagian aksinya memenuhi GOAL): RULE-LIST ← semua role yang aksinya memenuhi GOAL
2. Selama RULE-LIST belum kosong dan GOAL belum terbukti: **MONITOR**(rule pertama/berikutnya dari RULE-LIST), lalu hapus rule itu dari RULE-LIST
3. Jika tidak bisa disimpulkan: **request GOAL** (tanyakan ke user)

MONITOR(GOAL)

1. Uji proposisi pertama di bagian kondisi RULE
2. Jika parameter sudah diketahui: bila proposisi benar, lanjut ke proposisi berikutnya. Bila salah, RULE tidak bisa dieksekusi
3. Jika parameter belum diketahui: panggil FINDOUT(parameter)
4. Jika semua proposisi benar, eksekusi bagian aksi RULE

Intro to Learning and Supervised Learning

[Intro to Learning and Supervised Learning](#)
- [Konsep inti](#)
- [Problem Solving Agent & Knowledge Based Agent](#)
- [Knowledge acquisition bottleneck](#)
- [KBA with learning = Learning Agent](#)
  - [Arsitektur learning agent](#)
- [Desain learning element](#)
- [Jenis learning berdasarkan feedback](#)
- [Supervised learning](#)
- [Curve fitting, bias dan variance](#)

Konsep inti

- Sampai bagian KBS/RBS, pengetahuan agen harus **dimasukkan manual** oleh perancang
- **Learning** memungkinkan agen belajar sendiri dari pengamatan, sehingga bisa bekerja di lingkungan yang awalnya tidak diketahui.
- **Supervised learning** adalah belajar fungsi dari **pasangan input-output (data berlabel)** untuk memprediksi label data baru.

Problem Solving Agent & Knowledge Based Agent

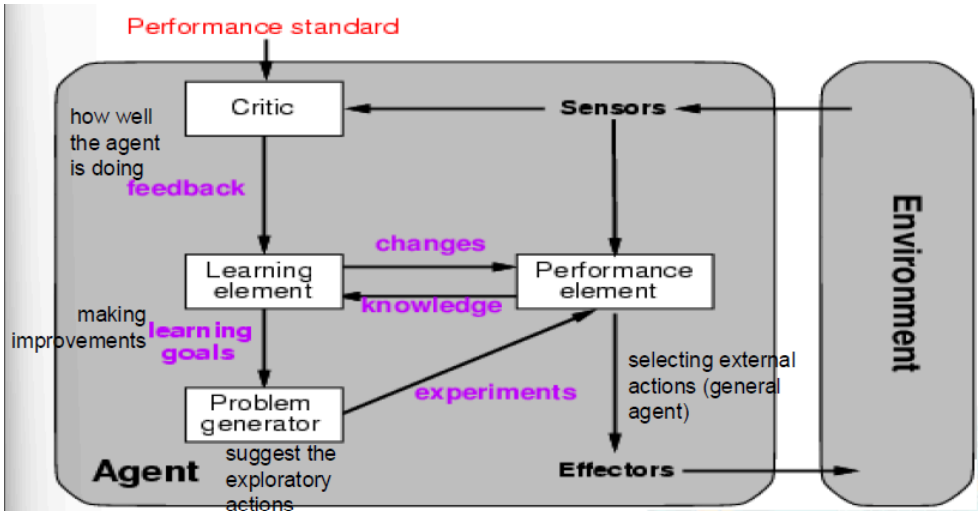
	Problem-solving agent	**Knowledge-based agent (KBA)**
Desain	Formulasi masalah → cari solusi → eksekusi	Knowledge base + inference engine
Asumsi dunia	Dinamika dunia diketahui, state terbatas (*finite*), deterministik, ada nilai state optimal	Dinamika dunia diketahui, deterministik. Slide menulis state **infinite**
Pengetahuan	-	Akuisisi **manual**: elicitation + representation

Knowledge acquisition bottleneck

Alasan perlunya learning: akuisisi pengetahuan manual itu lambat dan sulit.

- **Knowledge elicitation**:
  - **lambat**, dan
  - **pakar sering tidak mampu mengungkapkan** pengetahuan yang dimilikinya.
- **Knowledge representation**:
  - terjadi **knowledge mismatch**, yaitu perbedaan struktur pengetahuan pakar dengan struktur pengetahuan program.

KBA with learning = Learning Agent



- Dulu: KB kosong, perancang men-TELL kalimat satu per satu sampai agen tahu cara beroperasi. Dengan learning: agen **belajar dari pengamatan**.
- Learning memungkinkan agen beroperasi di lingkungan yang **awalnya tidak diketahui** dan **meningkatkan kinerja pada tugas mendatang** setelah mengamati dunia.

Arsitektur learning agent

Komponen	Peran (dari label di slide)
Performance element	**Memilih aksi eksternal** (agen umum): menerima persepsi dari sensors, mengeluarkan aksi lewat effectors

Critic	Menilai **seberapa baik agen bekerja** terhadap **performance standard**, dan memberi **feedback**
Learning element	**Membuat perbaikan**: mengubah (*changes*) performance element berdasarkan feedback dan pengetahuan, serta menetapkan *learning goals*
Problem generator	**Menyarankan aksi eksploratif** (*experiments*)
Sensors dan Effectors	Hubungan agen dengan **Environment**

Mengapa perlu learning:

1. **Lingkungan tidak diketahui**: perancang tidak mahatahu, agen tidak tahu dinamika dunia.
2. **Metode konstruksi sistem**: agen dipaparkan ke kenyataan, bukan perancang yang menuliskannya.
3. **Memodifikasi mekanisme keputusan agen** agar kinerja meningkat (belajar dari observasi, umpan balik untuk bertindak lebih baik nanti).

Desain learning element

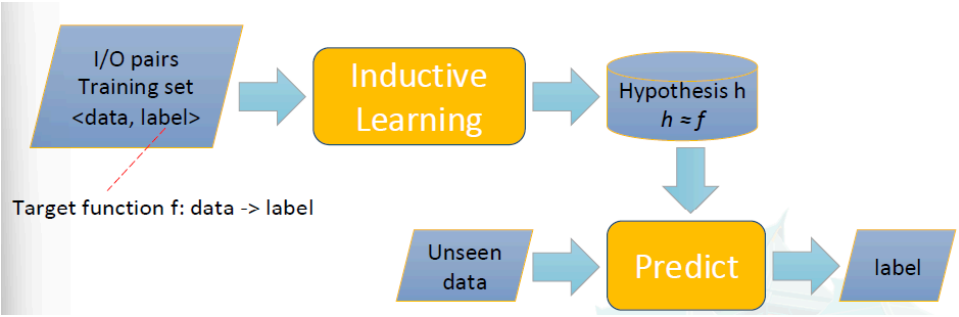
- **Learning** didefinisikan sebagai perubahan pada sistem yang bersifat adaptif, sehingga sistem mengerjakan tugas dari populasi yang sama **lebih efisien dan efektif** pada kesempatan berikutnya.
- **Empat hal yang menentukan desain learning element:**
    1. **Komponen** performance element mana yang akan dipelajari.
    2. **Ketersediaan pengetahuan awal** (*prior knowledge*).
    3. **Feedback** apa yang tersedia untuk mempelajari komponen tersebut.
    4. **Representasi** apa yang dipakai untuk komponen.

Jenis learning berdasarkan feedback

Jenis	Definisi	Contoh

Unsupervised (tanpa feedback)	Diberi contoh **tanpa label**, mendeteksi **cluster** input yang berpotensi berguna	Mengenali hari lalu lintas baik/buruk
Supervised	Diberi contoh **pasangan input-output**, mempelajari **fungsi** pemetaan input ke output	Keputusan mengerem
Reinforcement	Belajar dari serangkaian **reinforcement (reward atau punishment)**. Agen sendiri yang memutuskan aksi sebelumnya mana yang paling bertanggung jawab	-

Supervised learning

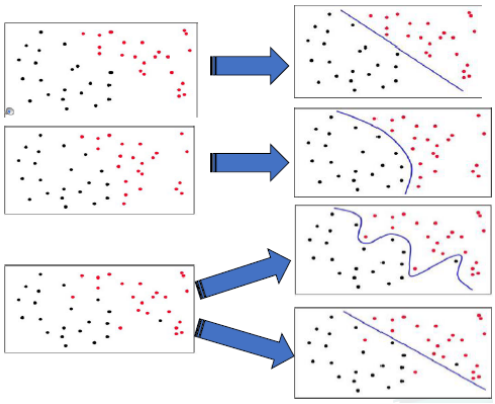


- **Input:** *training set* berpasangan <data, label>, ditulis <x, y>.
- **Target function:** f: x ⟶ y (tidak diketahui).
- **Masalah:** cari **hipotesis h** sehingga **h ≈ f**.
- **Proses: inductive learning**, yaitu mempelajari fungsi/aturan umum (mungkin tidak tepat) dari contoh input-output yang spesifik. **Bukan** analytical atau deductive learning.
- **Keluaran:** h (deskripsi kelas). Fungsi f bisa **stokastik**: P(Y | x).
- **Pemakaian:** mengklasifikasikan **data yang belum terlihat (unseen data)** berdasarkan deskripsi kelas.

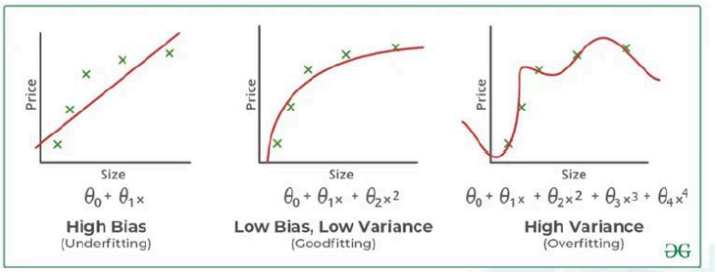
Dua jenis masalah:

	Classification	**Regression**
Domain y	**Himpunan nilai terbatas** (boolean, kelas)	**Bilangan** (suhu, harga saham)

Curve fitting, bias dan variance



gambar yang sama menunjukkan **beberapa hipotesis berbeda** yang bisa dipasang pada data yang sama (garis lurus, kurva, kurva berlekuk-lekuk)



Model	Bentuk	Label di slide
Garis lurus	$\theta_0 + \theta_1 x$	**High bias (underfitting)**
Polinomial derajat 2	$\theta_0 + \theta_1 x + \theta_2 x^2$	**Low bias, low variance (good fitting)**
Polinomial derajat 4	$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$	**High variance (overfitting)**

- **bias** tinggi berarti model terlalu sederhana sehingga gagal menangkap pola (*underfitting*).
- **Variance** tinggi berarti model terlalu menyesuaikan diri dengan data training, termasuk noise, sehingga kinerjanya turun pada data baru (*overfitting*)